



Università degli Studi di Salerno

Dipartimento di Ingegneria dell'Informazione ed Elettrica e
Matematica Applicata (DIEM)

Documento di progetto - Gruppo 6

Corso: Software Architecture Design

Docenti: Apicella Andrea, Ritrovato Pierluigi



Your favourite vibes

Versione 2

Membri

Autorino Luigi

Chirico Emanuel

Crisci Chiara

Graziuso Emanuela

1 Introduzione e panoramica

La presente relazione ha lo scopo di descrivere l'architettura e le funzionalità principali di '**VibeFlow**', una piattaforma desktop per la gestione e la riproduzione di tracce musicali. Il sistema mira a migliorare l'esperienza utente attraverso funzionalità avanzate di organizzazione, personalizzazione e controllo dinamico delle playlist e delle tracce.

Gli obiettivi funzionali principali del progetto sono:

- consentire la gestione completa delle tracce musicali;
- permettere la creazione e modifica dinamica delle playlist;
- supportare differenti modalità di riproduzione;
- garantire aggiornamenti in tempo reale dell'interfaccia;
- introdurre meccanismi di personalizzazione automatica;
- migliorare usabilità e flessibilità operativa.

Il documento è organizzato come segue: la Sezione 2 descrive i requisiti e come questi sono stati suddivisi in **User stories** dal team; la Sezione 3 descrive **l'architettura del sistema** e la scomposizione modulare in quattro domini (Track, Playlist, Playback Engine, View & Eventi); la Sezione 4 illustra la progettazione dell'interfaccia utente con wireframe e flussi di interazione.

VibeFlow è sviluppato in **Java 23.0.2** sfruttando **JavaFX** per l'interfaccia grafica. La scelta ricade su queste tecnologie per l'eccellente supporto ai design pattern utilizzati e per la capacità di gestire in modo nativo la scena grafica tramite thread dedicati alla riproduzione multimediale.

2 Requisiti e User Stories

Dall'analisi delle **epiche** sono state delineate le principali funzionalità alle quali il sistema deve assolvere e, una volta individuate le singole **user story**, queste sono state divise in vari task, visibili chiaramente dalla tabella sottostante.

In particolare, il sistema deve consentire la **gestione completa del ciclo di vita delle tracce musicali**, includendo operazioni di **creazione**, **modifica** ed **eliminazione** (ID_1, ID_2, ID_3). Ogni traccia è caratterizzata da metadati obbligatori (titolo, autore, genere, durata, anno) e da tag visivi opzionali assegnabili dinamicamente (ID_21). Inoltre *VibeFlow* deve **supportare la creazione e gestione di playlist manuali** (ID_5, ID_6, ID_7, ID_8) e **playlist automatiche** generate alitmicamente sulla base di metadati quali genere (ID_19), anno (ID_20), frequenza d'ascolto (ID_18) e tag visivi (ID_22).

Il motore di riproduzione deve supportare **tre modalità** distinte: sequenziale (ID_12), shuffle (ID_13) e loop (ID_14), con aggiornamento in tempo reale del player di riproduzione della traccia durante la riproduzione (ID_15). Le operazioni di aggiunta e rimozione tracce dalla libreria devono essere reversibili tramite meccanismo Undo (ID_16, ID_17).

ID	User Story	Priorità	Story points
	Sprint 1		
ID_1	Crea Track	High	3
ID_2	Modifica Track	High	1
ID_3	Rimuovi Track	High	1
ID_4	Visualizza Libreria Track	High	3
ID_5	Crea Playlist	High	3
ID_6	Aggiungi Track a Playlist	High	1
ID_7	Rimuovi Track da Playlist	High	1
ID_8	Elimina Playlist	High	3
ID_9	Riproduci Track / Playlist	High	5
ID_10	Sospendi / Riprendi Riproduzione	High	5
ID_11	Skip Track	High	5

	Sprint 2		
ID_12	Riproduzione 'Sequenziale'	High	5
ID_13	Riproduzione 'Shuffle'	High	5
ID_14	Riproduzione 'Loop'	High	5
ID_15	Aggiornamento Real-Time Player	High	8
ID_16	Undo Aggiunta Traccia	Medium	5
ID_17	Undo Rimozione Traccia	Medium	5
	Sprint 3		
ID_18	Playlist MostPlayed	Medium	3
ID_19	Playlist Genre	Medium	3
ID_20	Playlist Year	Medium	3
ID_21	Assegna Tag Visivi	Medium	2
ID_22	Playlist Tag Visivi	Medium	3
ID_23	Riordina Brani Playlist	Low	3
ID_24	Rinomina Playlist	Low	1
			TOT
			82

Il progetto è stato pianificato in **tre sprint** seguendo la metodologia Agile Scrum. Le user stories sono state prioritzate secondo il modello MoSCoW (High/Medium/Low) e stimate in Story Points da un valore minimo di 1 ad un valore massimo di 8.

Il totale ammonta a 84 Story Point distribuiti come segue:

- **Sprint 1** (ID_1 – ID_11, ID_24): funzionalità core di gestione tracce, playlist e riproduzione base.
- **Sprint 2** (ID_12 – ID_17): modalità di riproduzione avanzate e aggiornamento real-time.
- **Sprint 3** (ID_18 – ID_23): playlist automatiche, tag visivi e operazioni di riordino.

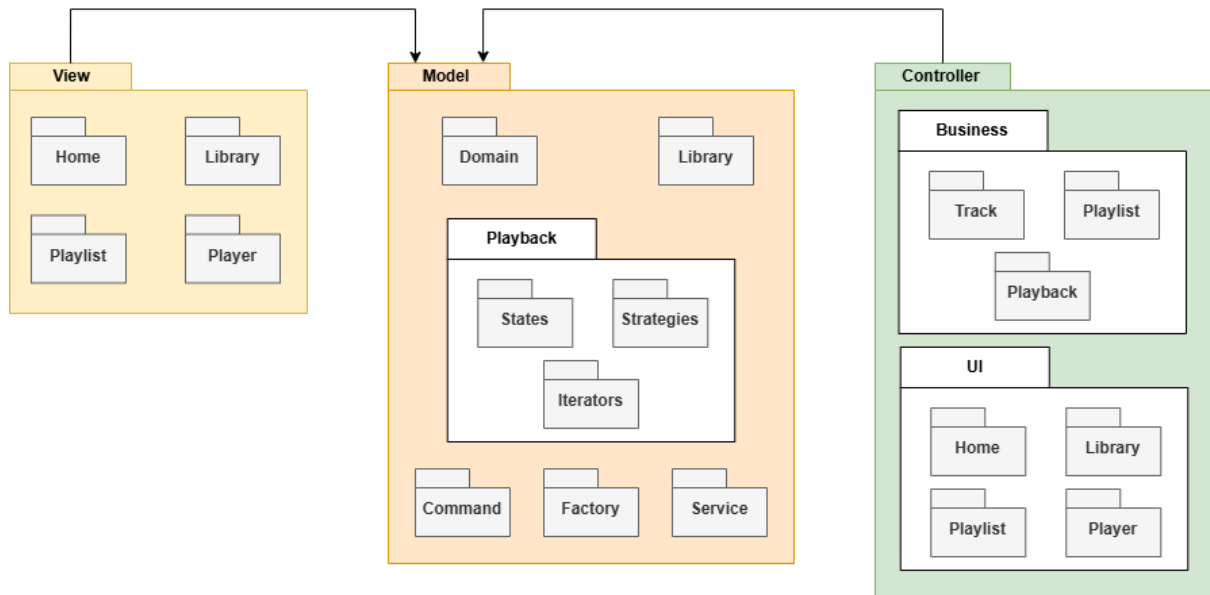
3 Architettura del Sistema

La presente sezione descrive l'architettura complessiva di VibeFlow, definendo i principali componenti del sistema e le scelte progettuali di alto livello adottate per tradurre i requisiti funzionali in una struttura software solida, modulare e manutenibile.

L'architettura del sistema **VibeFlow** si basa sul pattern architetturale *Model-View-Controller*. Al fine di garantire un'elevata coesione e un basso accoppiamento, il sistema adotta una rigorosa ripartizione dei compiti attraverso tre layer principali. Tale impostazione rispetta il **principio dell'inversione delle dipendenze**, in base al quale la logica di presentazione e di controllo dipendono dal modello dei dati, mentre quest'ultimo opera in totale isolamento. Inoltre, l'applicazione del MVC consente di migliorare la scalabilità del sistema, facilitare la manutenzione e rendere più agevole l'evoluzione delle singole componenti software.

- ❖ Il layer **Model** rappresenta il nucleo applicativo del sistema. Esso gestisce in via esclusiva la logica di business, l'accesso ai dati e lo stato dell'applicazione. Al suo interno è suddiviso in sei sotto-pacchetti funzionali. Il pacchetto *Domain* contiene le entità fondamentali del dominio applicativo. Il pacchetto *Library* gestisce i cataloghi di brani e playlist, integrando le interfacce del pattern Observer per una notifica degli eventi disaccoppiata. Il pacchetto *Playback* ospita il motore di riproduzione centrale; al suo interno, l'architettura isola i comportamenti complessi nei sub-package States per l'implementazione del pattern State, Strategies per le logiche di rimescolamento e ripetizione tramite il pattern Strategy, e Iterators per l'attraversamento delle code di riproduzione. Il pacchetto *Command* implementa l'omonimo pattern, incapsulando le azioni dell'utente in oggetti concreti per garantire la reversibilità delle operazioni. Il pacchetto *Factory* utilizza il pattern Factory Method per astrarre e automatizzare la creazione delle raccolte. Infine, il pacchetto *Service* fornisce componenti stateless dedicati alle attività di filtraggio del catalogo.
- ❖ Il layer **Controller** agisce da intermediario tra l'interfaccia utente e il modello. Per rispettare il *Single Responsibility Principle* all'interno del framework JavaFX, questo livello è stato strutturalmente scomposto in due macro-aree. L'area *Business* contiene i controller logici, i quali sono completamente indipendenti dall'interfaccia grafica: il loro ruolo è ricevere gli input, applicare le regole di dominio e orchestrare il Model delegando le modifiche di stato. L'area *UI* contiene invece i controller di presentazione associati ai file di markup. Il loro compito esclusivo è intercettare gli eventi generati dall'utente per inoltrarli ai controller di business e, contemporaneamente, agire da osservatori nei confronti del Model per aggiornare in tempo reale gli elementi visivi.

- ❖ Il layer **View** è dedicato unicamente alla presentazione passiva. Seguendo il *principio Open-Closed*, questo livello non contiene logica applicativa né file sorgente Java, ma è preposto all'organizzazione strutturale dei file di markup e dei fogli di stile. La sua organizzazione interna in sotto-pacchetti riflette in modo simmetrico la struttura dell'area UI del Controller. Questa scelta progettuale assicura una completa flessibilità nella sostituzione o nell'evoluzione del comparto grafico senza comportare alcun impatto sull'infrastruttura logica del sistema.

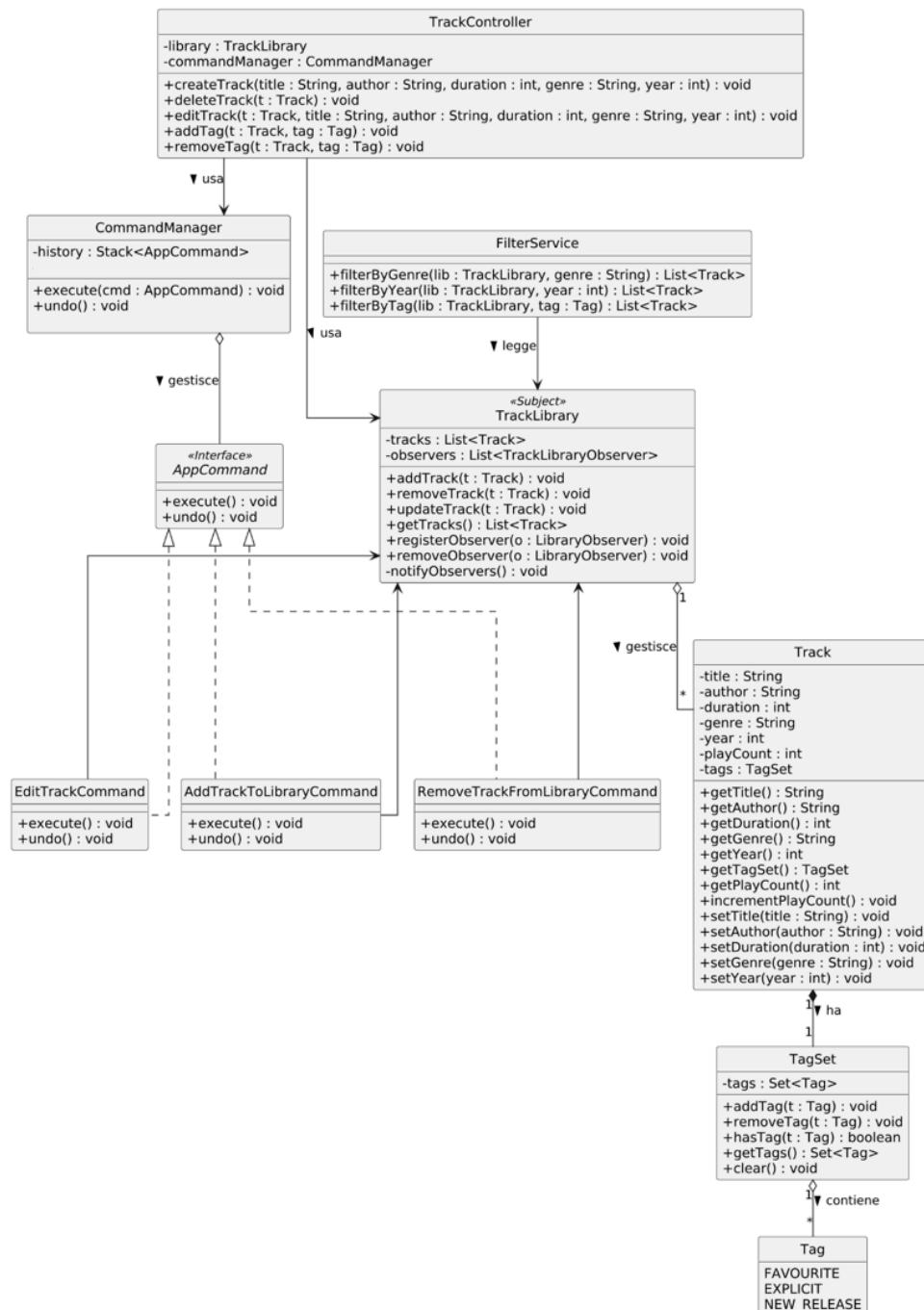


Per facilitare la comprensione dell'architettura e mettere in evidenza la natura modulare di VibeFlow, il sistema è stato suddiviso in moduli funzionali distinti. In particolare, l'analisi è stata organizzata attorno alle principali entità core del sistema: *Track*, *Playlist* e *Playback Engine*, a cui si aggiunge un quarto modulo dedicato alla gestione della presentazione e alla sincronizzazione degli eventi.

Nello specifico, la trattazione è articolata nei seguenti moduli:

- **Modulo 1** → Analisi dell'entità *Track*
- **Modulo 2** → Analisi dell'entità *Playlist*
- **Modulo 3** → Analisi dell'entità *Playback Engine*
- **Modulo 4** → Analisi delle *View* e degli *Observer*

Modulo 1: Track



Il modulo Track gestisce il ciclo di vita delle tracce musicali applicando in modo sistematico i principi SOLID e i pattern canonici per massimizzare coesione e minimizzare accoppiamento tra i componenti.

Il **TrackController** rappresenta l'unico punto di accesso per le operazioni CRUD sulla libreria, delegando l'esecuzione effettiva al **CommandManager** tramite il pattern **Command**.

Ogni operazione modificante viene incapsulata in un oggetto concreto che implementa l'interfaccia **AppCommand**, la quale espone i metodi **execute()** e **undo()**: le implementazioni previste sono **AddTrackToLibraryCommand**, **RemoveTrackFromLibraryCommand** ed **EditTrackCommand**.

Il **CommandManager** mantiene una struttura **Stack<AppCommand>** – history – realizzando un meccanismo completo di undo senza che il controller conosca i dettagli implementativi delle singole operazioni.

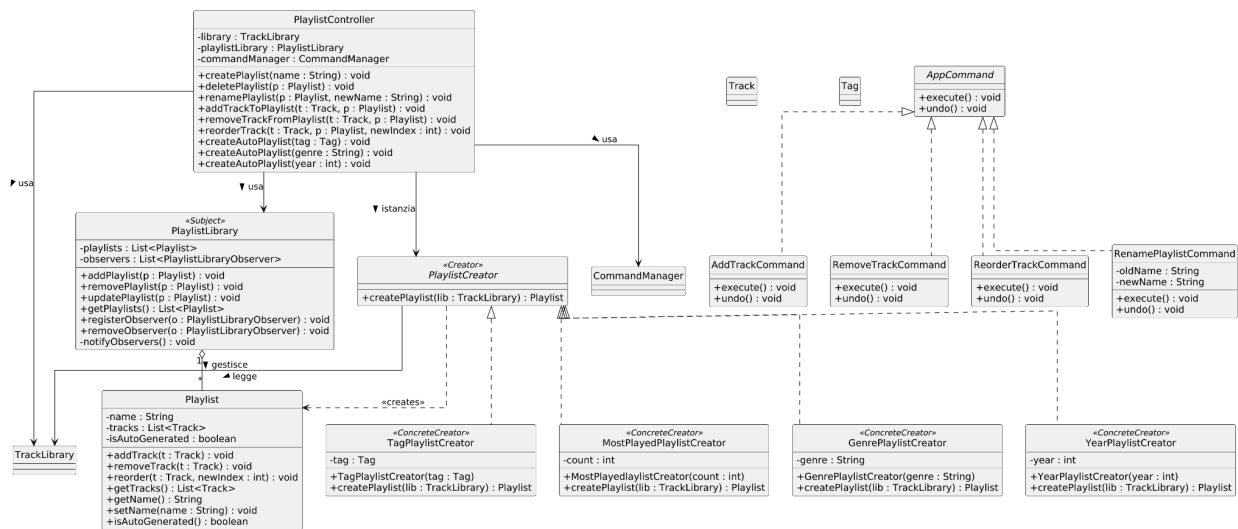
La **TrackLibrary**, marcata con lo stereotipo «Subject», adotta il pattern **Observer**: ogni operazione modificante sulla collezione innesca automaticamente la notifica a tutti i componenti registrati tramite **notifyObservers()**, garantendo la sincronizzazione in tempo reale dell'interfaccia utente, senza dipendenze dirette tra il layer di dominio e il layer di presentazione. La **TrackLibrary** dipende così dall'astrazione dell'interfaccia **Observer** e non dalle sue implementazioni concrete.

L'entità **Track** incapsula i metadati del brano e delega interamente la gestione dei tag alla classe **TagSet**, che aggrega i valori dell'enumerazione **Tag** (**FAVOURITE**, **EXPLICIT**, **NEW_RELEASE**) esponendo un'interfaccia dedicata.

Questa decomposizione isola la logica dei tag in un componente autonomamente testabile e consente di estendere il vocabolario dei tag senza modificare **Track**, rispettando il principio O del SOLID (*Open/Closed Principle*).

Il **FilterService** è un componente stateless che opera esclusivamente in lettura sulla **TrackLibrary** tramite **filterByGenre()**, **filterByYear()** e **filterByTag()**. L'assenza di stato interno e la dipendenza dalla sola interfaccia pubblica della libreria lo rendono utilizzabile in qualsiasi contesto, indipendente dall'ordine di invocazione e facilmente sostituibile senza impatto sul resto del sistema.

Modulo 2: Playlist



Il modulo **Playlist** gestisce il ciclo di vita delle playlist: creazione, rinomina, eliminazione, aggiunta e rimozione di tracce, riordinamento e generazione automatica, nonché i meccanismi di annullamento delle operazioni.

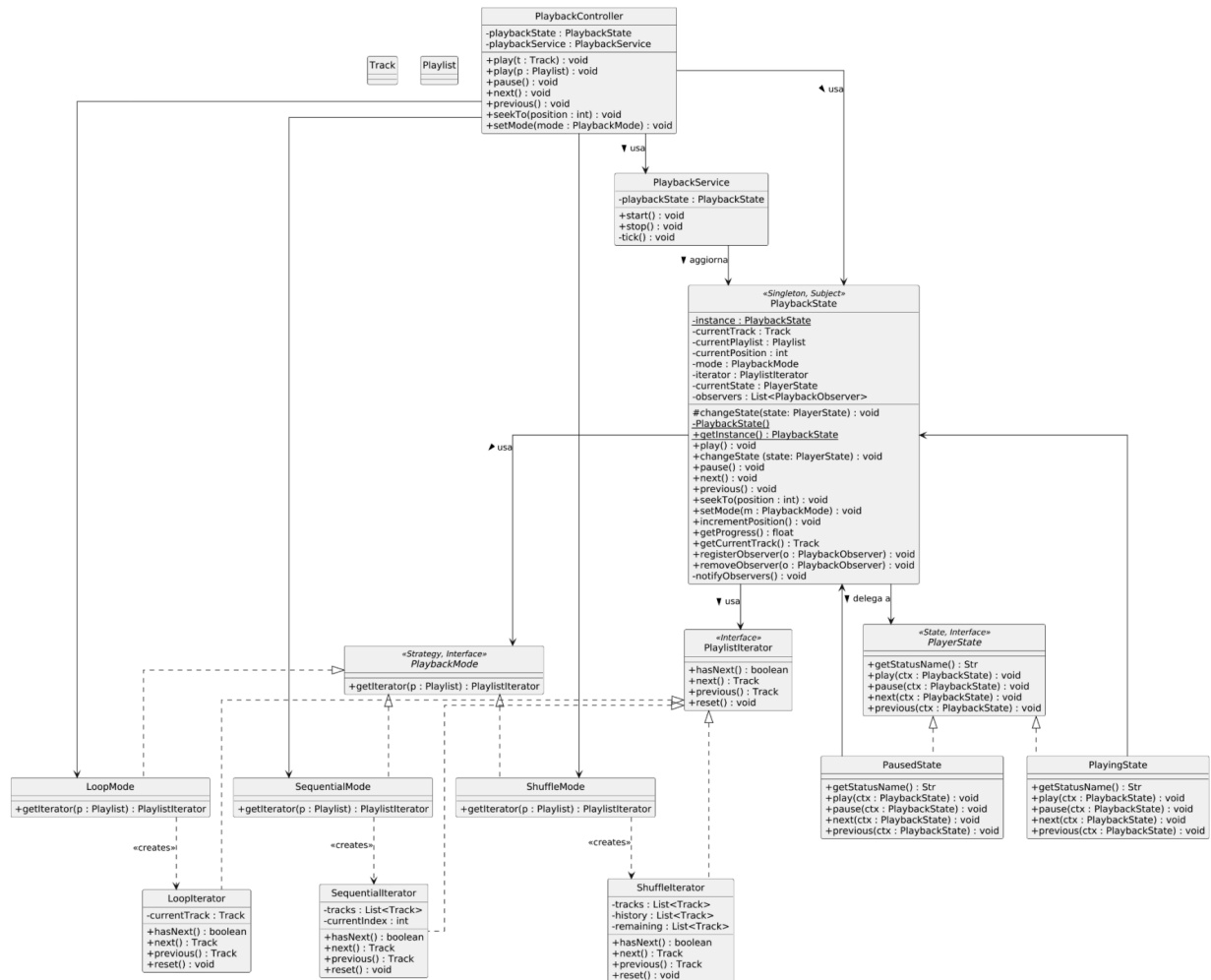
Il **PlaylistController** funge da punto di accesso unico per tutte le operazioni sulle playlist, delegando al **CommandManager** tramite il pattern **Command** ogni operazione modificativa: **AddTrackCommand**, **RemoveTrackCommand**, **ReorderTrackCommand** e **RenamePlaylistCommand** implementano l'interfaccia **AppCommand**, esponendo i metodi **execute()** e **undo()**. Questa architettura consente di mantenere uno stack di operazioni reversibili, senza introdurre dipendenze dirette tra il controller e la logica di dominio.

La **PlaylistLibrary**, indicata con «Subject», adotta il pattern **Observer**: ogni modifica alla collezione di playlist innesca la notifica automatica a tutti i componenti registrati.

La creazione delle playlist automatiche è astratta tramite il pattern **Factory Method**. La classe astratta **PlaylistCreator** dichiara il metodo **createPlaylist(lib : TrackLibrary) : Playlist**, implementato da quattro **ConcreteCreator**: **GenrePlaylistCreator**, **YearPlaylistCreator**, **TagPlaylistCreator** e **MostPlayedPlaylistCreator**.

Il **PlaylistController** seleziona il creator appropriato e invoca il Factory Method, rimanendo completamente indipendente dai criteri di filtraggio specifici di ciascuna tipologia. Questo consente di estendere i tipi di playlist automatica senza modificare il controller, rispettando il principio O del SOLID (*Open/Closed Principle*).

Modulo 3: Playback



Il modulo del motore di riproduzione rappresenta il nucleo architetturale più articolato di VibeFlow, integrando quattro pattern distinti in modo coordinato e sinergico. La complessità di questo modulo riflette la natura intrinsecamente stateful del dominio della riproduzione musicale, in cui molteplici componenti devono reagire in tempo reale a transizioni di stato, variazioni di posizione e cambiamenti di modalità.

La classe **PlaybackState**, contrassegnata con gli stereotipi «Singleton» e «Subject», costituisce il fulcro dell'intero modulo. L'adozione del pattern **Singleton** risponde a un preciso vincolo architetturale: lo stato del player caratterizzato da traccia corrente, posizione, modalità di riproduzione, iteratore attivo, deve essere unico e globalmente consistente. Il metodo statico **getInstance()** assicura che un'unica istanza venga creata e condivisa per l'intera durata del ciclo di vita dell'applicazione. Contestualmente, in qualità di «Subject» del pattern **Observer**, **PlaybackState** mantiene una lista di **PlaybackObserver** registrati e li notifica tramite **notifyObservers()** ad ogni variazione rilevante: questo meccanismo abilita l'aggiornamento in tempo reale della *progress bar*, dei metadati della traccia

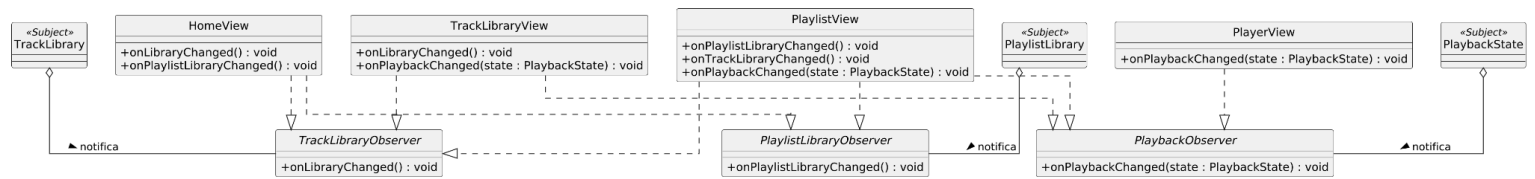
corrente e di qualsiasi altro componente dell'interfaccia interessato, preservando un disaccoppiamento tra il modello di riproduzione e le sue rappresentazioni visive. Il **PlaybackController** funge da entry point esterno per il layer di presentazione, esponendo le operazioni di alto livello (**play()**, **pause()**, **next()**, **previous()**, **seekTo()**, **setMode()**) e delegando l'esecuzione al **PlaybackService**, il quale gestisce il ciclo di aggiornamento interno tramite il metodo **tick()** e aggiorna lo stato tramite **PlaybackState**.

La gestione delle transizioni tra gli stati di riproduzione è affidata al pattern **State**, implementato tramite l'interfaccia «State, Interface» **PlayerState**, realizzata concretamente da **PlayingState** e **PausedState**. **PlaybackState** delega le operazioni comportamentali (**play()**, **pause()**, **next()**, **previous()**), allo stato corrente tramite il metodo **changeState()**, eliminando costrutti condizionali nel contesto e rendendo il comportamento del player estendibile senza modificarne il codice, in piena aderenza con il principio O del SOLID (*Open/Closed Principle*). L'aggiunta di un nuovo stato di riproduzione richiederebbe esclusivamente l'implementazione di una nuova classe concreta, senza alterare la logica esistente.

Per la selezione del brano successivo nella coda, il sistema adotta il pattern **Strategy** tramite l'interfaccia «Strategy, Interface» **PlaybackMode**, specializzata nelle implementazioni concrete **SequentialMode**, **ShuffleMode** e **LoopMode**. Ciascuna strategia è responsabile della propria logica di attraversamento della playlist tramite il metodo **getIterator(p : Playlist) : PlaylistIterator**, istanziando il corrispondente iteratore concreto – rispettivamente **SequentialIterator**, **ShuffleIterator** e **LoopIterator** – che implementa l'interfaccia «Interface» **PlaylistIterator**.

L'utilizzo del pattern **Iterator** si discosta dalla definizione canonica GoF, in cui è la collezione stessa a fornire il proprio iteratore. Delegare a **Playlist** la generazione di iteratori con logiche diverse avrebbe violato il **Single Responsibility Principle**, introducendo nella classe una dipendenza diretta dalle modalità di riproduzione. La responsabilità è pertanto affidata alla **Strategy** attiva, che istanzia l'iteratore corrispondente mantenendo **Playlist** come entità puramente contenitiva.

Modulo 4: View & Eventi



Il modulo **View & Eventi** rappresenta il layer di presentazione di VibeFlow e il sistema di notifiche che lo tiene sincronizzato con i modelli sottostanti. Il suo scopo architetturale non è aggiungere logica di business, ma **ricevere e reagire** agli aggiornamenti di stato prodotti dagli altri tre moduli : **Track**, **Playlist** e **Playback Engine**, in modo reattivo, disaccoppiato e in tempo reale.

Il pattern cardine dell'intero modulo è l'**Observer**, declinato in tre istanze parallele e indipendenti, ciascuna collegata a un Subject distinto.

Le tre interfacce Observer definite nel sistema sono:

- **TrackLibraryObserver**, che espone il metodo **onLibraryChanged()**. Viene notificata ad ogni modifica strutturale della TrackLibrary e consente alle View interessate di aggiornare la propria visualizzazione della libreria musicale.
- **PlaylistLibraryObserver**: espone il metodo **onPlaylistLibraryChanged()**. Viene notificata quando la PlaylistLibrary subisce variazioni, consentendo alle View di riflettere immediatamente lo stato aggiornato delle raccolte.
- **PlaybackObserver**: espone il metodo **onPlaybackChanged(...)**. Riceve in ingresso direttamente l'oggetto PlaybackState aggiornato, trasportando con sé tutti i dati rilevanti: traccia corrente, posizione di avanzamento, modalità attiva e stato di riproduzione. Questo design evita chiamate aggiuntive al modello per recuperare le informazioni necessarie all'aggiornamento della UI.

Ciascuna View implementa esattamente le interfacce Observer pertinenti alla propria funzione, senza sottoscrivere a notifiche di cui non necessita. Questa scelta rispetta l'**Interface Segregation Principle**: nessuna View è forzata a implementare metodi che non utilizzerebbe, e ogni interfaccia Observer è minimale e coesa.

- **HomeView**: implementa **TrackLibraryObserver** e **PlaylistLibraryObserver**. Reagisce ai cambiamenti di entrambe le librerie perché la home page aggrega informazioni provenienti da entrambi i domini.
- **TrackLibraryView**: implementa **TrackLibraryObserver** e **PlaybackObserver**. La prima sottoscrizione la mantiene sincronizzata con la lista delle tracce disponibili; la seconda le consente di evidenziare visivamente il brano in riproduzione all'interno dell'elenco, senza dover interrogare attivamente il player.

- **PlaylistView**: implementa tutte e tre le interfacce. È la view con il perimetro di notifica più ampio perché la schermata della playlist deve aggiornare il contenuto della raccolta, riflettere eventuali modifiche alle tracce contenute e indicare quale brano è in riproduzione.
- **PlayerView**: implementa **PlaybackObserver**. È la View più focalizzata del sistema: riceve lo stato aggiornato del player ad ogni tick del motore di riproduzione e aggiorna la barra di avanzamento, il contatore del tempo trascorso e i metadati della traccia corrente.

4 Progettazione dell'interfaccia utente

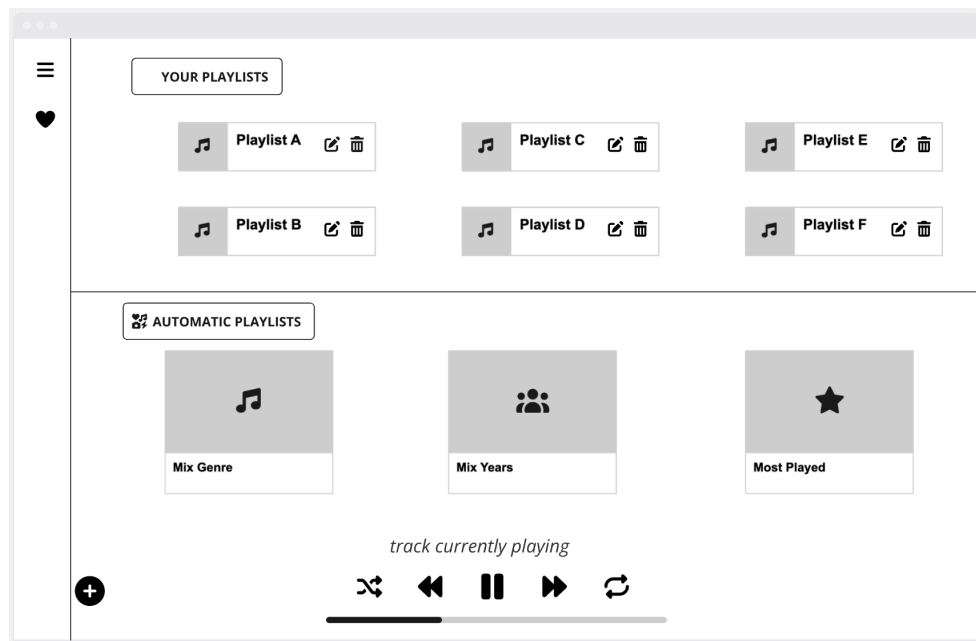
La presente sezione descrive la progettazione dell'interfaccia utente dell'applicazione VibeFlow. L'obiettivo è definire il **modo in cui l'utente interagisce con il sistema**, illustrando la struttura visiva delle schermate e i flussi di navigazione.

I *Wireframe* presentati in questa sezione sono stati realizzati con **Miro** e costituiscono il riferimento visivo per l'implementazione. Ogni schermata è descritta indicando il suo scopo, i componenti che la compongono e le User Stories che soddisfa.

4.1 Descrizione Schermate

- **Homepage**

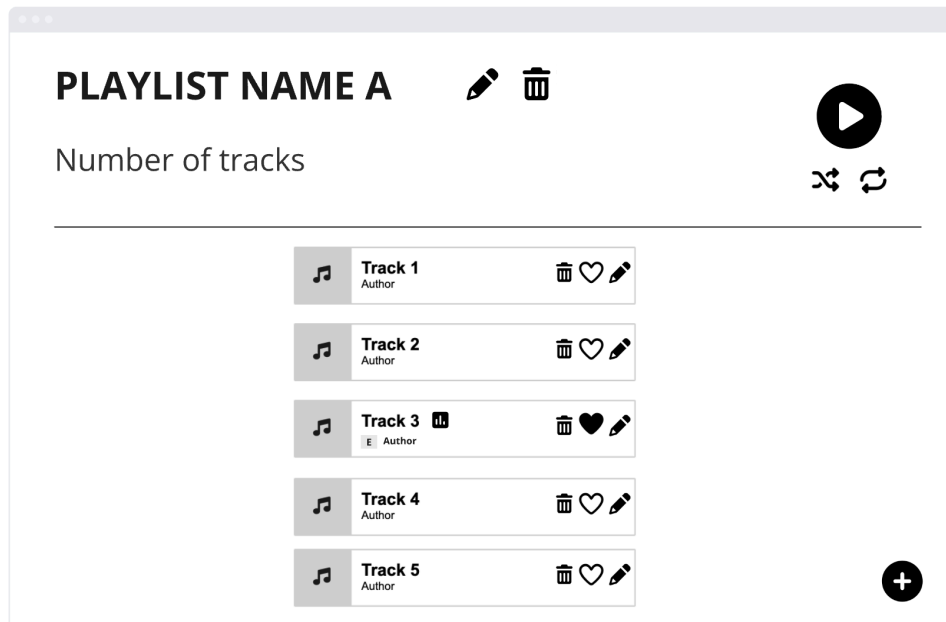
La homepage di VibeFlow costituisce il punto di accesso principale alle funzionalità di gestione e riproduzione. L'interfaccia è stata progettata separando nettamente le collezioni create dall'utente da quelle generate automaticamente dal sistema, mantenendo al contempo i controlli di riproduzione sempre accessibili.



- **Icona Menu:** serve per aprire la sezione **"All Tracks"**, consentendo all'utente di accedere alla libreria completa di tutti i brani creati nel sistema per visualizzarne i dettagli o effettuare operazioni di gestione.
- **Pulsante "+":** dedicato all'attivazione rapida dei flussi di creazione di nuove playlist.
- **Sezione "Your Playlists":** consiste in una griglia di elementi che rappresentano le raccolte create manualmente dall'utente. Ogni card include icone di azione rapida (matita e cestino) per le operazioni di gestione inline, descritte in dettaglio nella sezione [*Playlist Details*](#).
- **Sezione "Automatic Playlists":** sezione dedicata alle raccolte generate alitmicamente dal sistema per facilitare la scoperta dei contenuti. Include:
 - **Mix Genre:** Playlist basata sul raggruppamento per genere musicale.
 - **Mix Years:** Playlist basata sull'anno di pubblicazione dei brani.
 - **Most Played:** Selezione basata sulla frequenza di ascolto dell'utente.
- **Media Player:** Barra di controllo della riproduzione fissa nella parte inferiore della finestra. Il comportamento dettagliato dei controlli è descritto nella sezione [*Track Details*](#).

- **Playlist Details**

Questa interfaccia è progettata per la gestione granulare del contenuto della raccolta, permettendo operazioni di modifica sui metadati della playlist stessa, la gestione dei singoli brani contenuti al suo interno e il controllo della riproduzione focalizzato sulla coda selezionata.

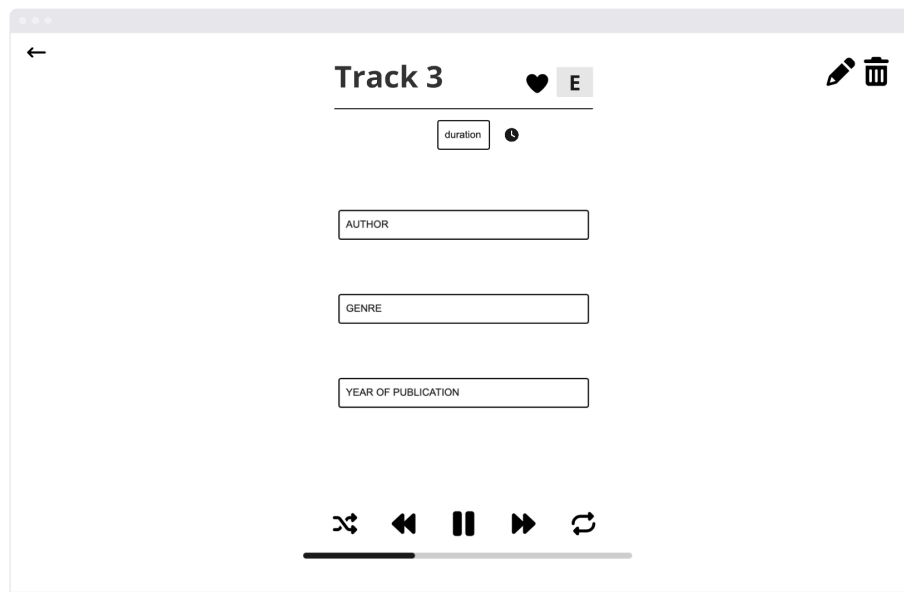


- **Intestazione Playlist:** mostra il nome della playlist corrente. Include:
 - **Icona Matita:** per la modifica del nome della playlist.
 - **Icona Cestino:** per l'eliminazione dell'intera playlist dal sistema.
- **Contatore brani:** Sotto il titolo, un'etichetta indica il "Number of tracks", fornendo un feedback immediato sulla dimensione della playlist.
- **Pannello di Controllo Master:**
 - **Pulsante Play:** Avvia la riproduzione della playlist partendo dal primo brano o riprendere l'ascolto.
 - **Icone Shuffle e Loop:** Permettono di configurare la modalità di riproduzione prima di avviare la playlist. Il comportamento dettagliato di queste funzioni è descritto nella sezione Track Details.
- **Lista dei Brani:** Elenco verticale delle tracce incluse. Ogni riga contiene:
 - **Info Brano:** Titolo e Autore del brano.
 - **Tag Visivi:** Icone specifiche come il tag "E" (Explicit), generato automaticamente dal sistema, o icone di stato riproduzione.
 - **Strumenti di azione rapida:**
 - **Cestino:** rimuove il brano esclusivamente da questa playlist.
 - **Cuore (Preferiti):** permette di assegnare/rimuovere il tag "Preferito" al brano.
 - **Matita:** apre il modulo di modifica dei metadati del singolo brano.
- **Indicatore brano attivo:** un'icona segnala visivamente quale traccia è attualmente in esecuzione.

- **Pulsante "+":** dedicato esclusivamente all'aggiunta di nuovi brani all'interno di questa specifica playlist.

- Track Details

Questa interfaccia è dedicata all'ispezione di tutti i metadati di un brano.

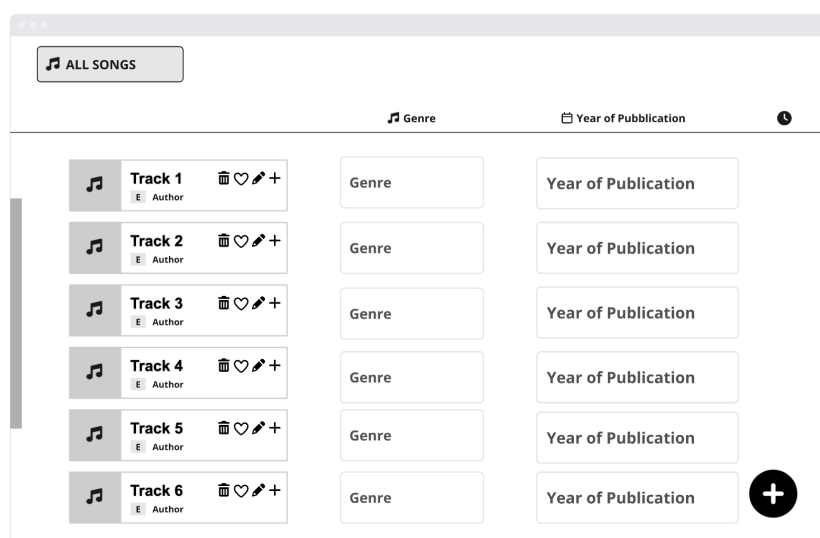


- **Pulsante 'indietro':** Permette all'utente di tornare alla schermata precedente senza interrompere la riproduzione in corso.
- **Header traccia:** mostra il titolo del brano. Accanto al titolo sono presenti i **Tag Visivi**:
 - **Icona cuore:** indica se il brano è tra i "Preferiti".
 - **Tags:** Segnala visivamente caratteristiche peculiari della traccia (Explicit, New Release, ecc.)
- **Pannello Gestione:**
 - **Icona matita:** per l'attivazione della modalità di modifica dei campi.
 - **Icona Cestino:** per l'eliminazione della traccia dal sistema.
 - **Blocchi Informativi Metadati:** Area centrale che espone i metadati.
- **Media Player Persistente:** costituisce la sezione operativa di base, posizionata in modo fisso nel footer di questa schermata, funge da terminale di controllo per il brano in esame o per la coda di riproduzione attiva.
- **Pannello dei Controlli di Flusso:** situato centralmente, permette la gestione dinamica del motore di riproduzione attraverso icone standardizzate:

- **Shuffle:** Attiva/disattiva la logica di rimescolamento della coda. Se attiva, il sistema seleziona la traccia successiva in modo non sequenziale senza alterare l'ordine fisico della playlist.
- **Skip Backward:** Implementa la logica di controllo temporale avanzata; se premuto nei primi 10 secondi del brano, carica la traccia precedente, altrimenti riavvia il brano corrente dall'istante iniziale.
- **Play/Pause:** Il pulsante centrale cambia icona in base allo stato interno del Player. Gestisce la sospensione del flusso audio e la sua ripresa dallo stesso istante temporale.
- **Skip Forward:** Interrompe la riproduzione corrente per avviare immediatamente la traccia successiva nella coda di riproduzione.
- **Loop:** Cicla due diverse modalità di ripetizione (traccia singola oppure intera playlist), istruendo il sistema a riprodurre il primo brano della playlist al termine dell'ultimo brano della stessa.
- **Sistema di Feedback Real-time:**
 - **Barra di Avanzamento:** Una linea di progresso orizzontale che riflette la posizione temporale esatta del brano. Si riempie gradualmente da sinistra verso destra man mano che il tempo trascorre.

- All tracks

La schermata **All Tracks** è la vista tabellare completa di tutte le tracce caricate nel sistema. Il layout è organizzato per colonne, facilitando la scansione visiva e la comparazione tra i diversi brani in base a genere, anno e durata.



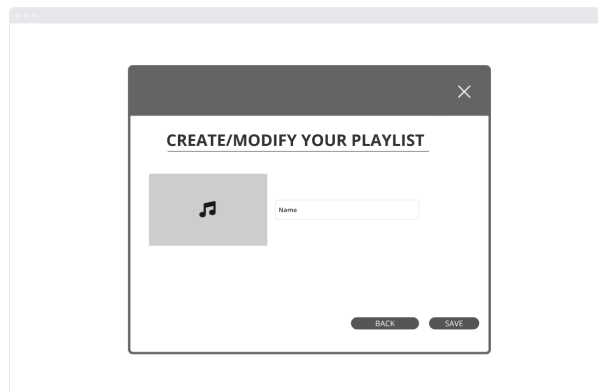
ALL SONGS				Genre	Year of Publication	
	Track 1		Author	Genre	Year of Publication	
	Track 2		Author	Genre	Year of Publication	
	Track 3		Author	Genre	Year of Publication	
	Track 4		Author	Genre	Year of Publication	
	Track 5		Author	Genre	Year of Publication	
	Track 6		Author	Genre	Year of Publication	

- **Header dei Metadati:** Una riga di intestazione che identifica le colonne informative: genere, anno, durata.

- **Pulsante "+" Globale:** Floating Action Button per l'aggiunta di nuovi brani alla libreria globale. Questa azione crea una traccia a livello di sistema.

- **Playlist creation**

Questa interfaccia funge da modulo universale per la gestione dei metadati delle playlist.



- **Campo di Input "Name":** Unico campo di testo interattivo dedicato all'inserimento o alla modifica del nome della playlist.
 - **Validazione Obbligatorietà:** Il sistema impedisce il salvataggio se il campo è vuoto.
 - **Validazione Univocità:** Il sistema controlla che il nome non sia già stato assegnato a un'altra playlist esistente.
- **Pulsante "BACK":** Permette di chiudere il modulo e tornare alla visualizzazione precedente senza apportare alcuna modifica al database.
- **Pulsante "SAVE":** Trigger principale per la persistenza dei dati. Alla pressione, il sistema esegue i controlli di validazione prima di confermare l'operazione e aggiornare l'interfaccia principale.

- **Track creation**

Questa interfaccia è dedicata alla gestione dei dati di ogni brano del sistema. È progettata per garantire l'integrità del database, obbligando l'utente all'inserimento di tutti i metadati necessari per il corretto funzionamento del sistema.

CREATE/MODIFY YOUR TRACK

Add these fields

Title

Author

Genre

Duration

Year of Publication

BACK SAVE

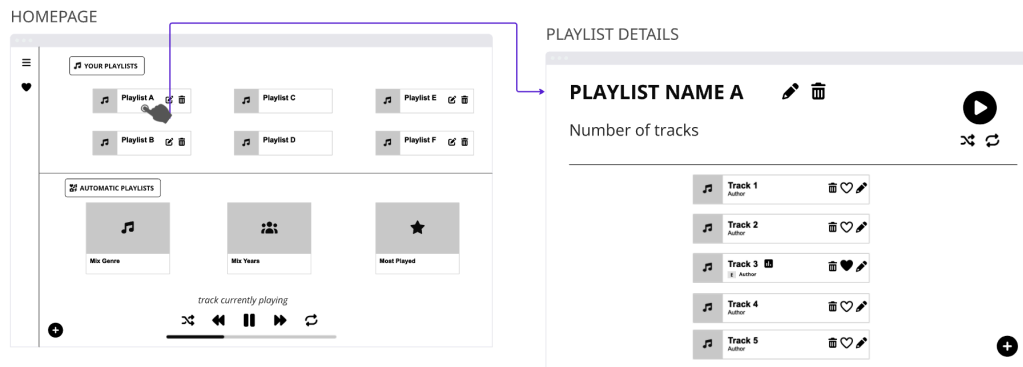
- **Inserimento Dati:** Un set di cinque campi di testo obbligatori (Title, Author, Genre, Duration - *valore numerico espresso in secondi* - , Year of Publication).
- **Pulsante "BACK":** Funziona secondo la stessa logica descritta nella sezione Playlist Creation: annulla l'operazione e riporta l'utente alla vista precedente senza alcuna modifica al database.
- **Pulsante "SAVE":** Azione primaria posizionata in basso a destra. Segue la stessa logica di validazione descritta nella sezione Playlist Creation, con le seguenti specifiche aggiuntive per i campi traccia:
 - In caso di successo i dati vengono salvati e l'utente viene reindirizzato alla libreria;
 - In caso di errore viene generato un Alert di sistema che segnala il campo non valido.

4.2 Descrizione flussi d'interazione

La presente sezione descrive i principali flussi di interazione tra le schermate dell'applicazione, illustrando come l'utente naviga da una vista all'altra in risposta alle proprie azioni. Ogni flusso è accompagnato dal Wireframe corrispondente, che mostra visivamente la transizione tra le schermate coinvolte.

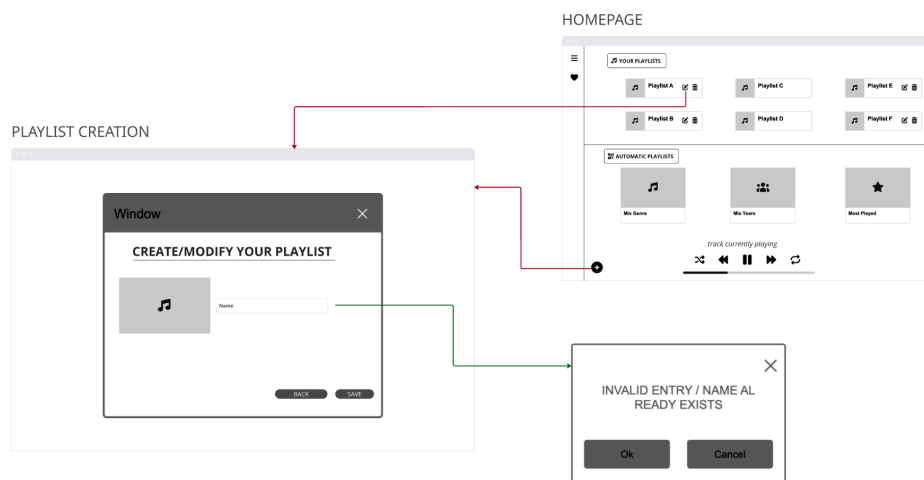
- Homepage - Playlist Details

L'utente, dalla schermata homepage, seleziona una playlist (manuale o automatica) e il sistema reindirizza alla schermata di Playlist Details corrispondente, permettendogli di avere una visualizzazione completa delle tracce appartenenti alla playlist selezionata.



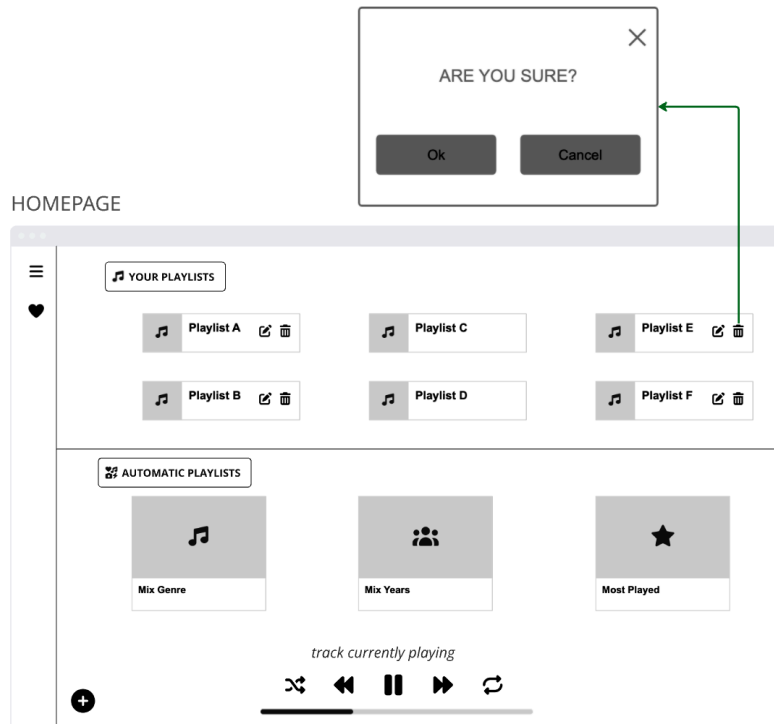
- Homepage - Playlist Creation

L'utente, dalla homepage, preme il pulsante di aggiunta (+) oppure di modifica di una playlist e si apre il dialog di creazione/modifica playlist. In caso di mancata compilazione dei campi obbligatori oppure di utilizzo di un nome già esistente, il sistema mostra un messaggio di errore.



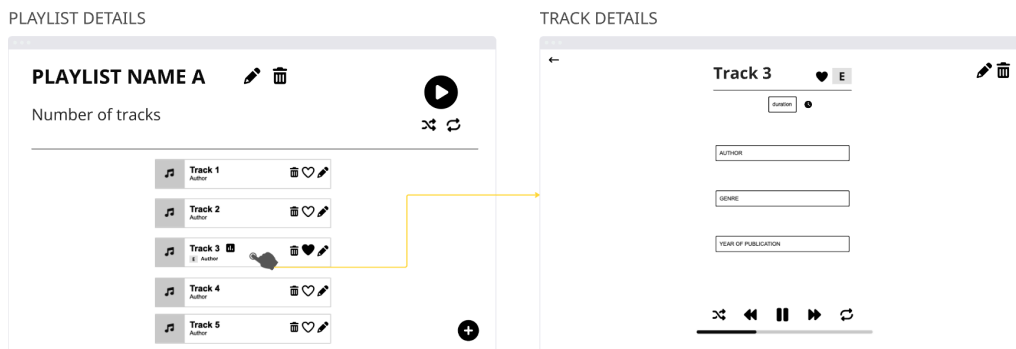
- Homepage - Dialog

L'utente, dalla homepage, preme l'icona di eliminazione (cestino) su una playlist e il sistema mostra un dialog di conferma "Are you sure?". Alla conferma, la playlist viene rimossa e l'interfaccia si aggiorna.



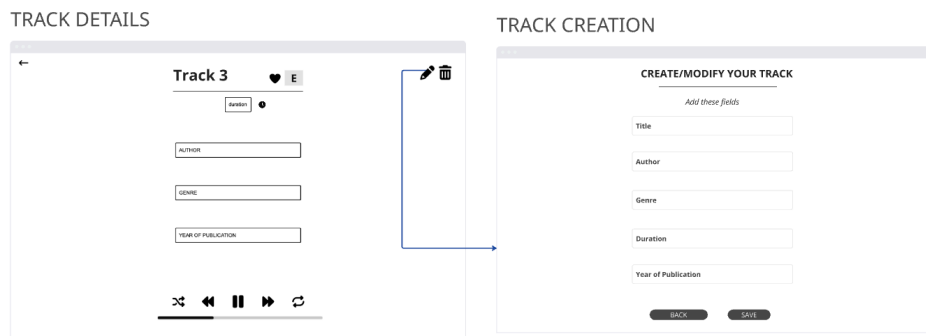
- Playlist Details - Track Details

L'utente, dalla schermata di Playlist Details, preme sul corpo di una riga (es. "Track 3") e il sistema reindirizza alla schermata di Track Details, da cui può visualizzare i metadati della traccia e controllare la riproduzione tramite i controlli del player integrati nella schermata.



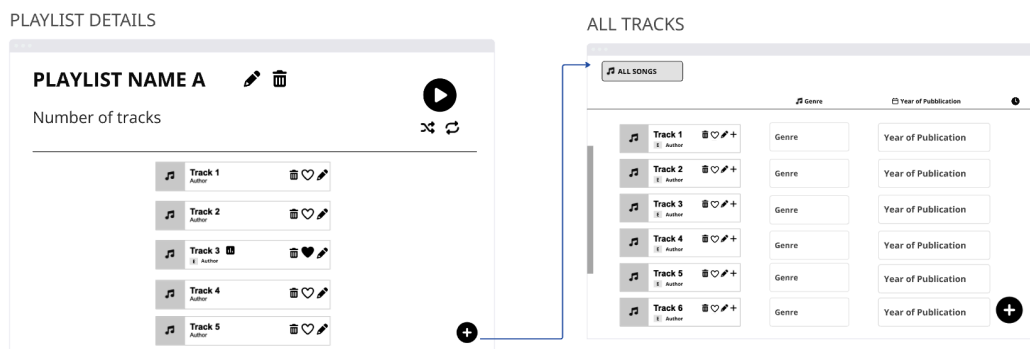
- Track Details - Track Creation

L'utente, dalla schermata Track Details, preme l'icona di modifica su una traccia esistente. Il sistema reindirizza alla schermata Track Creation precompilato con i metadati correnti della traccia selezionata. La logica di validazione e il comportamento in caso di errore seguono quanto descritto precedentemente.



- Playlist Details - All Tracks

L'utente, dalla schermata Playlist Details, preme sul bottone di aggiunta (+) per inserire una nuova traccia all'interno della playlist selezionata (es. "playlist name A"). Il sistema reindirizza alla schermata di visualizzazione di tutte le tracce, per consentire all'utente di selezionare una delle tracce da aggiungere.



- All tracks - Track Creation - Dialog

L'utente, dalla schermata All Tracks, ha la possibilità di creare una nuova traccia premendo il bottone di aggiunta (+). Il sistema reindirizza alla schermata di Track Creation e in essa l'utente inserisce tutti i campi obbligatori. Nel caso di errata o mancata compilazione dei campi previsti, viene mostrato un dialog di "Invalid Entry".

ALL TRACKS

The 'ALL TRACKS' interface features a sidebar with a button labeled 'ALL SONGS'. The main area contains a table with columns for track details and editing options. The table has two main sections: a list of tracks and a table for editing track details.

Track 1		Genre	Year of Publication
Track 1	Author	Genre	Year of Publication
Track 2	Author	Genre	Year of Publication
Track 3	Author	Genre	Year of Publication
Track 4	Author	Genre	Year of Publication
Track 5	Author	Genre	Year of Publication
Track 6	Author	Genre	Year of Publication

TRACK CREATION

The 'TRACK CREATION' interface is titled 'CREATE/MODIFY YOUR TRACK'. It includes a section 'Add these fields' with input fields for Title, Author, Genre, Duration, and Year of Publication. At the bottom, there are 'BACK' and 'SAVE' buttons.

An 'INVALID ENTRY' dialog box with a close button (X) and 'Ok' and 'Cancel' buttons.

Note aggiuntive:

Si specifica che le logiche di interazione associate alle icone di **modifica** e **rimozione** sono da considerarsi standardizzate per l'intero applicativo. Ogni occorrenza dell'icona di modifica attiva sistematicamente il relativo dialog di inserimento (per la gestione dei metadati), così come ogni pressione dell'icona di eliminazione è vincolata alla comparsa del dialog di conferma (per le operazioni distruttive), garantendo un comportamento coerente in ogni sezione.